

DATA301-DATA471: Data Science in Practice

Assignment 4

Leon Cai (300692424)

Victoria University of Wellington
September 2026

Exercise 1

1.1 Order statistics for the quartiles [10%]

Summary of the two articles. Nine common definitions of the sample quantile are compared in Wicklin (2017), and they all follow the same general form given in Hyndman and Fan (1996) (H&F): $\hat{Q}(p) = (1 - \gamma)x_{(j)} + \gamma x_{(j+1)}$, where $x_{(j)}$ and $x_{(j+1)}$ are the order statistics closest to p and γ is an interpolation weight, with j and γ both depending on p , the sample size n , and a parameter m that is different for each definition ($j = \lfloor np + m \rfloor$).

The nine definitions split into two groups. Types 1-3 are discontinuous, meaning they jump between values instead of changing smoothly, since they come directly from the empirical CDF, and they always give a value that already appears in the data (or the average of two adjacent values) – this group is plotted as a staircase in Wicklin (2017). Types 4-9 are continuous and interpolate between order statistics using a *plotting position* p_k , the probability assigned to order statistic $x_{(k)}$, and these six types split further by how p_k is derived: types 6, 7 and 8 use a *distribution-free* rule (p_k does not depend on the population distribution F), while type 9 is built to be almost unbiased only for QQ-plots against a Normal distribution, so it depends on F instead.

Each definition is also checked in Hyndman and Fan (1996) against six properties a good sample quantile should have (continuity in p , symmetry in the tails, uniqueness, and matching the usual sample median at $p = 0.5$), and only type 5 satisfies all six, but it's a compromise that was not built from an estimation argument. Types 6-9 each satisfy five of the six and have a clearer theoretical basis, and of these, H&F recommend type 8 as a general default because its estimate of $Q(p)$ is approximately median-unbiased for *any* F – types 6 and 7 don't have this unbiased property, and type 9 is unbiased only for the Normal case. There's also a practical point in Wicklin (2017) that matters here: the nine definitions agree closely once n is fairly large (the article uses $n = 100$), but differ more for small samples or when the data have large gaps between neighbouring values, which includes the $n = 5$ cells used in the Monte Carlo comparison below.

Choice used in this report. Type 8 is used throughout (the default for `lambda_iqr()` above, and everywhere it is called), for two extra reasons on top of H&F's own recommendation: the Exponential distribution used here is clearly skewed, unlike the Normal that type 9 targets, and the smallest sample size in the design, $n = 5$, is exactly where Wicklin (2017) shows the choice of definition matters most. And this choice isn't only about this section – $\hat{\lambda}_{IQR}$ gets used again in the bootstrap correction below, and in every replication of the Monte Carlo experiment further on, so a bad quantile definition here would carry through to those results too.

Software default. R's `quantile()` defaults to type 7, confirmed below. This comes from

S-Plus, whose `quantile()` used type 7 (Hyndman and Fan (1996), Section 4), and R inherited it. Type 7 is not the type 8 that H&F recommend.

```
quantile(1:10) # uses R's default type

 0%   25%   50%   75%  100%
1.00  3.25  5.50  7.75 10.00

identical(quantile(1:10), quantile(1:10, type = 7)) # confirms default = type 7

[1] TRUE
```

1.2 Bootstrap bias-corrected estimator [20%]

Derivation. The bias of $\hat{\lambda}_{IQR}$ would ideally be found by simulating many samples from the true $\text{Exponential}(\lambda)$ distribution and comparing the average of $\hat{\lambda}_{IQR}$ across those samples to the true λ , as covered in the Week 9 lecture, but in practice there is only one sample $\mathbf{X}$ and the true λ is unknown. Bootstrap gets around this by treating $\mathbf{X}$ itself as a stand-in for the population, and $\hat{\lambda}_{IQR}(\mathbf{X})$ as a stand-in for the unknown true value. For a fixed sample $\mathbf{X}$ of size n , draw B bootstrap resamples $\mathbf{X}_1^*, \dots, \mathbf{X}_B^*$ (sampling n values from $\mathbf{X}$ with replacement). The bootstrap estimate of the bias of $\hat{\lambda}_{IQR}$ is

$$\widehat{\text{bias}} = \frac{1}{B} \sum_{b=1}^B \hat{\lambda}_{IQR}(\mathbf{X}_b^*) - \hat{\lambda}_{IQR}(\mathbf{X}).$$

The first-order bias-corrected estimator subtracts this estimated bias:

$$\tilde{\lambda}(\mathbf{X}) = \hat{\lambda}_{IQR}(\mathbf{X}) - \widehat{\text{bias}} = 2\hat{\lambda}_{IQR}(\mathbf{X}) - \frac{1}{B} \sum_{b=1}^B \hat{\lambda}_{IQR}(\mathbf{X}_b^*).$$

```
lambda_ml <- function(x) 1 / mean(x)

lambda_iqr <- function(x, type = 8) {
  q <- quantile(x, probs = c(0.25, 0.75), type = type)
  log(3) / (q[[2]] - q[[1]])
}

lambda_tilde <- function(x, B = 200, type = 8) {
  n <- length(x)
  boot_est <- replicate(B, lambda_iqr(sample(x, n, replace = TRUE),
                                         type = type))
  # Small n: a bootstrap resample can tie at the quartiles (q3=q1), making
  # lambda_iqr(.) = Inf. Drop these before averaging so one Inf doesn't
  # dominate the correction.
  boot_est <- boot_est[is.finite(boot_est)]
  2 * lambda_iqr(x, type = type) - mean(boot_est)
}

x <- rexp(30, rate = 2)
c(ML = lambda_ml(x), IQR = lambda_iqr(x), bias_corrected = lambda_tilde(x))
```

ML	IQR	bias_corrected
1.562896	1.741311	1.814164

1.3 Monte Carlo comparison [20%]

Design. The structure follows the course’s own Monte Carlo tutorial (Frery 2026): for each situation $(\lambda, n) \in \Lambda \times N$, R independent replications are generated, each replication gives one sample $\mathbf{X}$ of size n and one value of each of the three estimators, and the MSE for that situation is the sample mean of $(\hat{\lambda} - \lambda)^2$ over the R replications.

$R = 5000$ is used for every situation here, straight from the Week 9 lecture, where the lecturer said 5000 replications per case is enough for an assignment this size, and that you only need to control R more tightly in much bigger experiments. It also leaves a decent margin on precision: the standard error of the Monte Carlo estimate of MSE is about $\sqrt{\widehat{\text{Var}}[(\hat{\lambda} - \lambda)^2]/R}$, and even at $n = 5$, where the estimators are most variable, $R = 5000$ keeps this small next to the MSE values being compared.

```
mc_mse <- function(lambda, n, R, B = 200, type = 8) {
  est <- matrix(NA_real_, nrow = R, ncol = 3,
               dimnames = list(NULL, c("ML", "IQR", "bias_corrected")))
  for (r in seq_len(R)) {
    x <- rexp(n, rate = lambda)
    est[r, ] <- c(lambda_ml(x), lambda_iqr(x, type = type),
                  lambda_tilde(x, B = B, type = type))
  }
  colMeans((est - lambda)^2)
}

lambda_grid <- c(0.1, 1, 10)
n_grid <- c(5, 30, 100)
R <- 5000

results <- do.call(rbind, lapply(lambda_grid, function(lam) {
  do.call(rbind, lapply(n_grid, function(n) {
    mse <- mc_mse(lam, n, R = R)
    data.frame(lambda = lam, n = n, R = R, t(mse), check.names = FALSE)
  }))
}))
results
```

	lambda	n	R		ML	IQR	bias_corrected
1	0.1	5	5000	6.350623e-03	1.256162e-02	1.869204e+01	
2	0.1	30	5000	3.995203e-04	1.002016e-03	1.242778e-03	
3	0.1	100	5000	1.080374e-04	2.360035e-04	2.788539e-04	
4	1.0	5	5000	5.684110e-01	9.374353e-01	4.749525e+03	
5	1.0	30	5000	3.783969e-02	9.543773e-02	1.185430e-01	
6	1.0	100	5000	1.065686e-02	2.300534e-02	2.690247e-02	
7	10.0	5	5000	6.177431e+01	9.638678e+01	7.003303e+04	
8	10.0	30	5000	4.012428e+00	9.753578e+00	1.229160e+01	
9	10.0	100	5000	1.081235e+00	2.446096e+00	2.822481e+00	

The same numbers are easier to compare as a plot, especially since `bias_corrected`’s MSE spans several orders of magnitude across the design – Figure 1 puts both axes on a log scale.

```
est_names <- c("ML", "IQR", "bias_corrected")
results_long <- do.call(rbind, lapply(est_names, function(est) {
  data.frame(lambda = results$lambda, n = results$n,
```

```

    estimator = est, mse = results[[est]])
}))

ggplot(results_long, aes(n, mse, color = estimator)) +
  geom_line() + geom_point() +
  scale_x_log10(breaks = c(5, 30, 100)) +
  scale_y_log10() +
  facet_wrap(~ lambda, labeller = label_both) +
  labs(x = "n (log scale)", y = "MSE (log scale)", color = "Estimator")

```

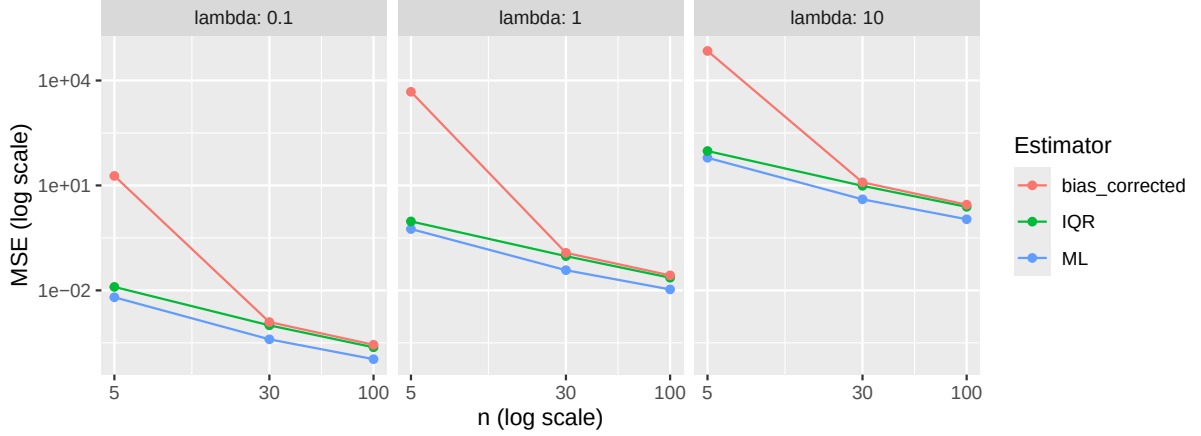


Figure 1: MSE of the three estimators across the (λ, n) design, log-log scale.

Discussion. $\hat{\lambda}_{ML}$ has the lowest MSE in every one of the nine cells. This matches theory. It is the maximum likelihood estimator, so it uses the full sample through the likelihood, and it has the smallest asymptotic variance among regular estimators. $\hat{\lambda}_{IQR}$ only uses two order statistics, so it should do worse. The MSE gap between the two stays in a similar range across the design: $\hat{\lambda}_{IQR}$'s MSE is roughly 1.6 to 2.5 times $\hat{\lambda}_{ML}$'s, at every (λ, n) combination, with no clear trend as n or λ change. This roughly matches the idea that the IQR estimator has a fixed *asymptotic* efficiency loss from throwing away most of the sample, but that result only has to hold as $n \rightarrow \infty$ – some of the spread in the ratio seen here could be genuine finite-sample behaviour rather than just Monte Carlo noise from a design that is too small to pin the ratio down exactly.

The bias-corrected estimator $\tilde{\lambda}$ is *worse* than the uncorrected $\hat{\lambda}_{IQR}$ at every single (λ, n) in the design, not only at $n = 5$. At $n = 100$, the correction makes MSE about 15-18% bigger than $\hat{\lambda}_{IQR}$, across all three λ values. At $n = 30$, the gap is about 24-26%. At $n = 5$, it is much worse: three to four orders of magnitude bigger. The reason is a bias-variance trade-off that goes the wrong way. $\hat{\lambda}_{IQR}$ is a nonlinear function (a reciprocal) of the sample's interquartile spread. A bootstrap resample can, by chance, have a smaller or larger spread than the original sample. After taking the reciprocal, and then doubling it in $\tilde{\lambda} = 2\hat{\lambda}_{IQR} - \hat{\lambda}_{IQR}^*$, this small chance difference turns into a large swing in the correction term. The variance this adds is bigger than the bias it removes, so MSE gets worse overall, and more so as n shrinks, because bootstrap resamples are then more likely to have an unusually small spread. So the first-order bootstrap correction does not help this estimator, at any of the sample sizes tested here.

All three estimators' MSE decreases roughly with $1/n$, matching the usual parametric convergence rate, and increases roughly with λ^2 : at $n = 5$, $\text{MSE}(\hat{\lambda}_{ML})/\lambda^2$ is about 0.64, 0.57, and 0.62 at $\lambda = 0.1, 1, 10$, close enough that the ratio looks roughly constant. That makes sense: $\hat{\lambda}$ is a rate, so squared error scales with λ^2 under a change of unit.

Exercise 2

`ausbeer` comes with the `fpp2` package. It is quarterly beer production data for Australia. Figure 2 shows the full series; the sections on frequency, trend, seasonality, variance and unusual observations all refer back to this same series, some directly from the plot and some from numbers computed later.

```
autoplot(ausbeer, color = "steelblue") +  
  labs(y = "Megalitres")
```

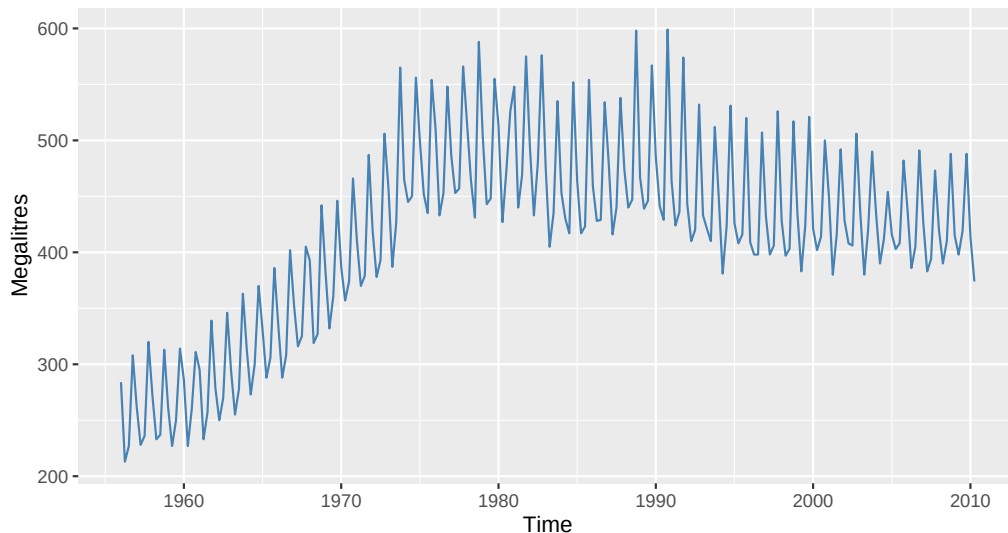


Figure 2: Quarterly Australian beer production, 1956 Q1–2010 Q2.

2.1 What is the frequency of the series? How can you determine this from the data and/or the time-series plot?

The frequency is 4 (quarterly data), and the series runs from 1956 Q1 to 2010 Q2, confirmed below by `frequency()`, `start()` and `end()`.

```
frequency(ausbeer)
```

```
[1] 4
```

```
start(ausbeer)
```

```
[1] 1956    1
```

```
end(ausbeer)
```

```
[1] 2010    2
```

Counting cycles by eye on the full 54-year series in Figure 2 is hard, since each year only takes up a few pixels, but zooming into a handful of years in Figure 3 makes the four-per-year pattern countable directly:

```
autoplot(window(ausbeer, start = 1985, end = 1990), color = "steelblue") +  
  labs(y = "Megalitres")
```

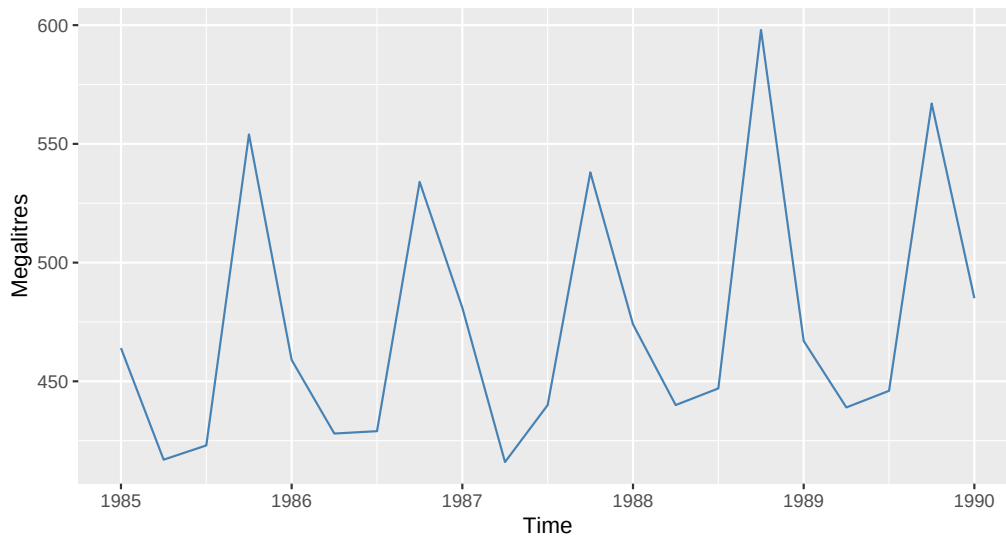


Figure 3: `ausbeer` zoomed into 1985–1990, showing four quarters per year.

2.2 Describe the overall behaviour of the series. Is there evidence of a long-term trend?

There isn't one single trend here, more like three phases. Production rises steadily from 1956 to around the mid-1970s (from about 260 to about 485 megalitres, nearly doubling), then plateaus and fluctuates around a peak from the mid-1970s to about 1990, then drops fairly sharply in the early 1990s (from about 490 down to about 445 in just three years) before levelling off at that lower level through the 2000s.

To check this against a straight line without fitting any model, Figure 4 joins the early-period average (1956-60) to the late-period average (2005-10) with a red line. If the trend were linear, the yearly average would stay close to this line all the way through. Instead it sits above it for about thirty-five years, from the mid-1960s to around 2000, by roughly 100-165 megalitres between 1970 and 1990, and only comes back to the line at the end. The yearly average, in blue, makes the three phases clearer than either the raw quarterly plot (Figure 2) or the straight line on their own.

```
ausbeer_df <- data.frame(time = as.numeric(time(ausbeer)),
                        value = as.numeric(ausbeer))
yearly <- aggregate(value ~ floor(time), ausbeer_df, mean)
names(yearly) <- c("year", "value")
yearly <- yearly[yearly$year < 2010, ] # 2010 only has 2 quarters, drop it

early <- mean(ausbeer_df$value[ausbeer_df$time < 1961])
late <- mean(ausbeer_df$value[ausbeer_df$time >= 2005])
t_early <- mean(ausbeer_df$time[ausbeer_df$time < 1961])
t_late <- mean(ausbeer_df$time[ausbeer_df$time >= 2005])
ausbeer_df$line <- early +
  (late - early) * (ausbeer_df$time - t_early) / (t_late - t_early)

ggplot(ausbeer_df, aes(time, value)) +
  geom_line(alpha = 0.5) +
  geom_line(aes(y = line), color = "red", linetype = "dashed") +
```

```
geom_line(data = yearly, aes(year + 0.5, value),
          color = "blue", linewidth = 1) +
labs(x = "Time", y = "Megalitres")
```

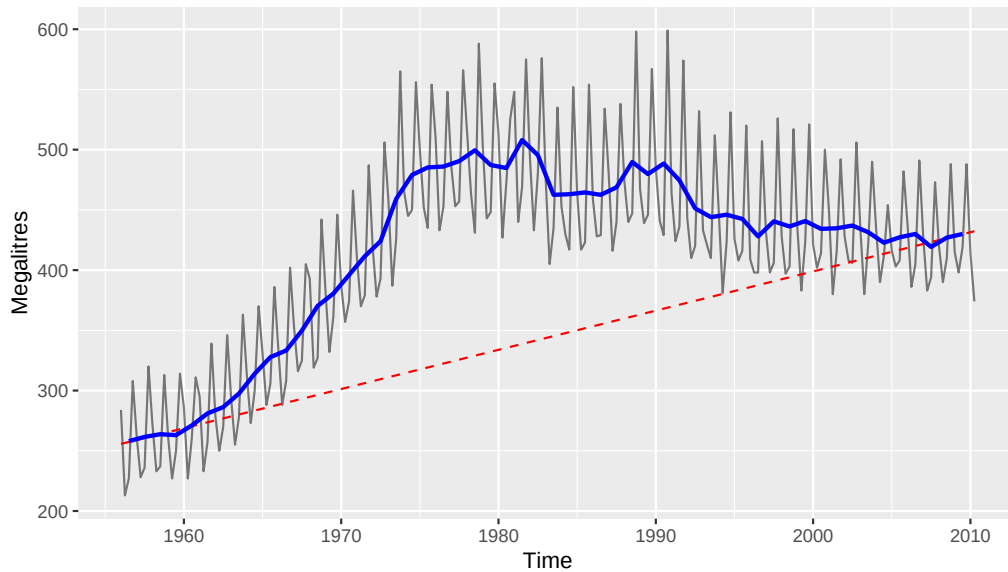


Figure 4: `ausbeer`'s yearly average (blue) against a straight line joining the early- and late-period averages (red).

2.3 Is there evidence of seasonality? If so, describe the seasonal pattern.

Yes, and it's a strong, regular pattern: every year has the same shape, low in Q2 and Q3 and high in Q4, confirmed by the quarter averages below. Q4 is clearly the highest quarter and Q2 the lowest.

```
quarter_means <- aggregate(as.numeric(ausbeer) ~ cycle(ausbeer), FUN = mean)
names(quarter_means) <- c("quarter", "mean")
quarter_means
```

quarter	mean
1	416.8182
2	372.6182
3	387.3704
4	485.4444

Figure 5 shows this isn't just an average effect: Q4's box sits above Q2's and Q3's with no overlap at all, and only a small overlap with Q1's, so Q4 being the high quarter is a consistent, year-on-year pattern rather than something driven by a few big years.

```
qdf <- data.frame(quarter = factor(cycle(ausbeer)),
                  value = as.numeric(ausbeer))
ggplot(qdf, aes(quarter, value, fill = quarter)) +
  geom_boxplot(outlier.shape = NA, alpha = 0.7, show.legend = FALSE) +
  geom_jitter(width = 0.08, alpha = 0.3, size = 0.6, show.legend = FALSE) +
  scale_fill_brewer(palette = "Set2") +
  labs(x = "Quarter", y = "Megalitres")
```

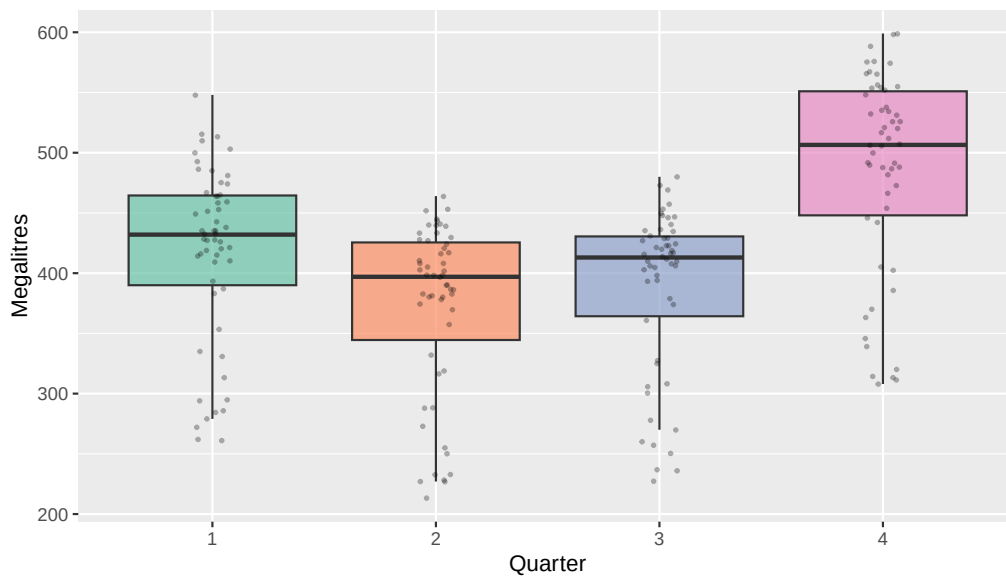


Figure 5: Distribution of `ausbeer` by quarter.

2.4 Does the variability appear constant over time? Explain what you see in the plot and whether a transformation might be appropriate.

Not really constant, but not in the usual direction. This shows up in Figure 2: the seasonal swings look visibly tighter in the 2000s than they did in the 1970s-90s, even though the level in that period isn't dramatically different. The breakdown below puts a number on it. The variability (measured here as the standard deviation within each period) actually *shrinks* over time, even though the level itself goes up and then back down:

```
breaks <- c(1955, 1970, 1980, 1990, 2000, 2011)
labels <- c("1956-70", "1971-80", "1981-90", "1991-2000", "2001-10")
period <- cut(ausbeer_df$time, breaks = breaks, labels = labels)
aggregate(value ~ period, data.frame(ausbeer_df, period),
           function(x) c(mean = mean(x), sd = sd(x)))
```

	period	value.mean	value.sd
1	1956-70	305.56140	56.76487
2	1971-80	464.97500	61.07225
3	1981-90	477.20000	53.73926
4	1991-2000	447.62500	54.90724
5	2001-10	427.85366	37.98523

The standard deviation doesn't track the mean in any simple way here: 1971-80 has the highest sd despite a middling mean, and 1981-90 has the highest mean despite one of the *lower* sds. The clearest pattern is at the far end, where the most recent period's spread (38) is distinctly smaller than every earlier period's (54-61), even though its own level isn't unusual. The relative variability (sd divided by mean) tells a similar story, highest early and lowest late, though not a clean monotonic slide in between. Either way, this is not the classic case for a log transform, where the size of the fluctuations is supposed to grow *with* the level. If anything the opposite shows up at the two ends of the series.

2.5 Are there any observations that you would regard as unusual? Identify them and explain why they attract attention.

Two things stand out, of different kinds. The first is structural: the whole pre-1975-ish period behaves differently from the rest of the series, both in level (much lower) and in relative variability (much higher, as shown above). Even the “outlier” points below the whiskers in Figure 5 are all from 1956-61, so they’re this same structural thing showing up again rather than scattered noise. This same split shows up in Hyndman and Athanasopoulos (2018), which restricts to `window(ausbeer, start = 1992)` before fitting any forecasting model to this dataset, since the earlier decades reflect a different, still growing market rather than the more settled pattern seen from the 1990s on.

The second is a genuine point-level anomaly: 1981 Q1 comes in at 548, well above its neighbouring Q1 values (453-515 across the surrounding decade). Comparing every quarter with the average of the same quarter in the three years either side (no model involved), it is the largest gap anywhere in `ausbeer`: 56 megalitres above its neighbours, about 3.8 times the typical size of that gap:

```
ausbeer_df$quarter <- as.numeric(cycle(ausbeer))
local_gap <- sapply(seq_len(nrow(ausbeer_df)), function(i) {
  near <- ausbeer_df$quarter == ausbeer_df$quarter[i] &
    abs(ausbeer_df$time - ausbeer_df$time[i]) <= 3 &
    seq_len(nrow(ausbeer_df)) != i
  if (sum(near) < 4) return(NA)
  ausbeer_df$value[i] - mean(ausbeer_df$value[near])
})
ausbeer_df$time[which.max(abs(local_gap))]
```

```
[1] 1981
```

```
max(abs(local_gap), na.rm = TRUE) / sd(local_gap, na.rm = TRUE)
```

```
[1] 3.799203
```

2.6 Plot the series by season (for example, using a seasonal plot). What does this reveal that is less obvious in the ordinary time-series plot?

```
ggseasonplot(ausbeer)
```

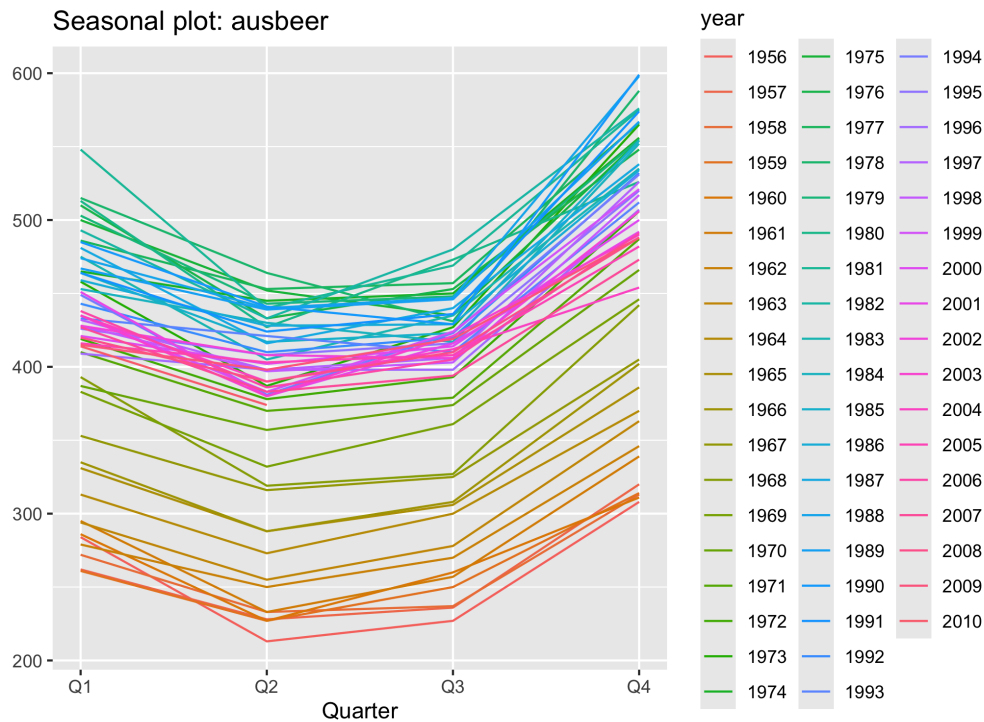


Figure 6: Seasonal plot of `ausbeer`, one line per year.

Figure 6 puts every year on its own line across the four quarters. Compared to the ordinary time plot (Figure 2), this makes a few things easier to see:

- Almost every year has the same up-down-up shape: climbing to a peak at Q4 without exception, and usually dipping lowest at Q2 (though Q3 is the lowest quarter in six of the 54 full years, like 1978 and 1990).
- The lines are stacked in the same order as the trend. The earliest (coolest-coloured) years sit lowest, the lines climb up through the 1970s-80s, and the most recent years sit back down again.
- Comparing the same quarter across different years is much easier here than in the ordinary time plot, where the years are stretched out left to right instead of stacked on top of each other.
- The 1981 anomaly from section 2.5 is visible here too: its line starts at Q1 clearly above every neighbouring year's line, standing out precisely because the rest of that colour range clusters so tightly together.

2.7 Plot the seasonal subseries. Which quarters have the highest and lowest typical values? Is the seasonal pattern stable over time?

```
p_subseries <- ggsubseriesplot(ausbeer)
p_subseries$layers[[1]]$aes_params$colour <- "steelblue"
p_subseries$layers[[2]]$aes_params$colour <- "red"
p_subseries + labs(y = "Megalitres")
```

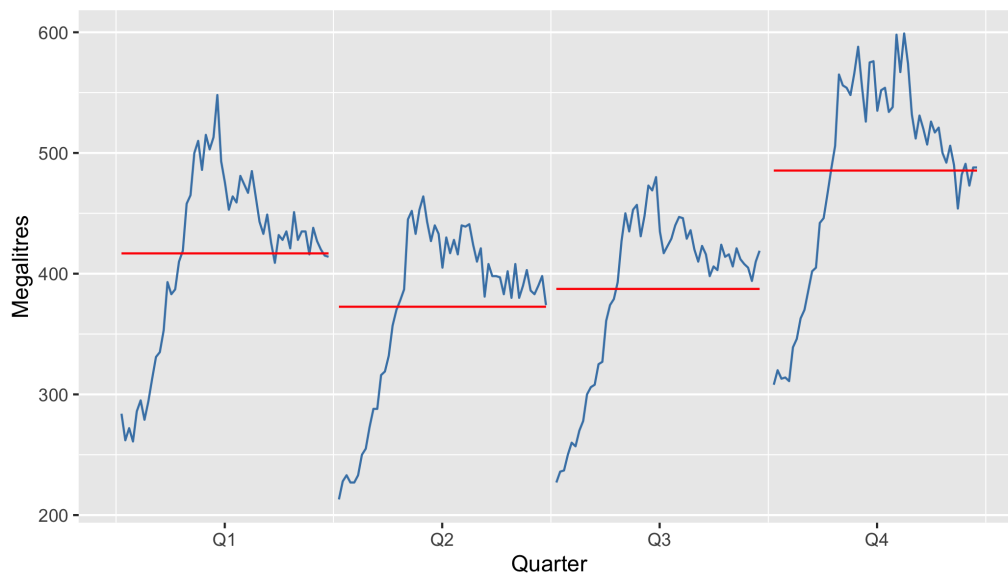


Figure 7: Seasonal subseries plot of `ausbeer`, one panel per quarter.

Figure 7 splits the series into one panel per quarter, keeping the years in time order within each panel (unlike the jittered points in Figure 5) and marking each quarter's mean in red:

- Unlike the boxplot, keeping the time order visible means each panel still traces its own version of the rise-then-plateau-then-decline shape from the trend. So the *level* within a quarter is not stable over time, since the seasonal and trend components aren't independent of when in the series you're looking.
- It also confirms two things from section 2.3: Q4 has the highest typical value and Q2 the lowest, and the seasonal *pattern* (which quarter is high and which is low) looks stable, not flipping or fading out in any panel.

2.8 Would you detrend or transform the series before modelling it? Give a visual argument for your answer. Do not fit a model.

Detrending, yes. Figure 8 plots the gap between the raw series and the same straight line as in Figure 4: just two averages connected by a line, not a fitted model. If a straight line described the trend, the gap would only wobble around zero with the seasonal swings. Instead it forms a clear hump: close to zero at the two ends, where the line is anchored, then well above zero through the 1970s-80s (on a yearly average, roughly 100-165 megalitres) before coming back down. That's the same rise-plateau-fall shape as the trend itself, so a straight line clearly doesn't describe the series, and the trend would need to come out before it's modelled as something stationary.

```
ausbeer_df$gap <- ausbeer_df$value - ausbeer_df$line

ggplot(ausbeer_df, aes(time, gap)) +
  geom_line(color = "steelblue") +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  labs(x = "Time", y = "Value minus the straight line")
```

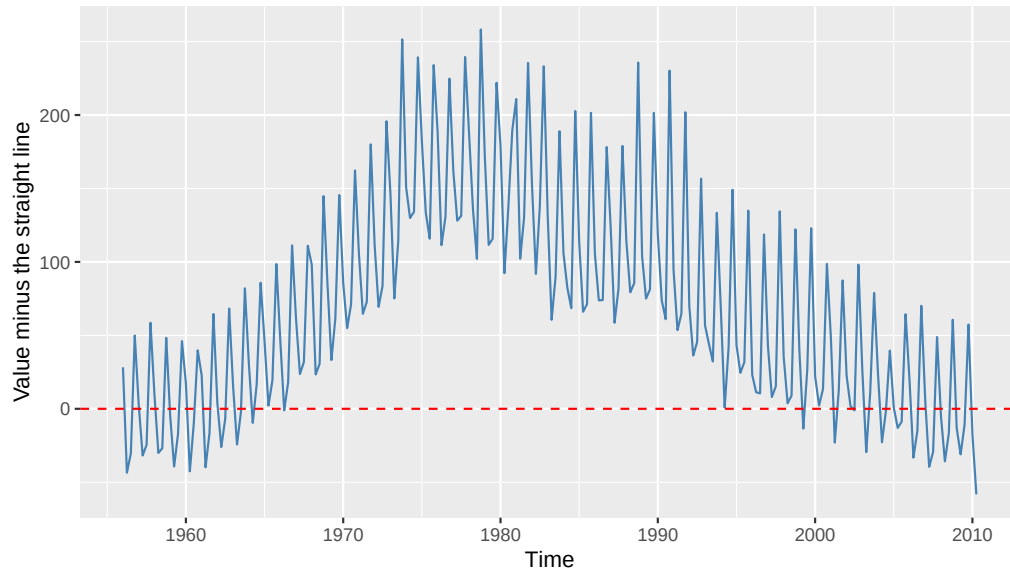


Figure 8: Gap between **ausbeer** and the straight line in Figure 4.

A variance-stabilising transform (log or Box-Cox) is less clearly justified. The usual visual argument for a log transform is that spread grows with the level, and neither Figure 2 nor the breakdown in section 2.4 really show that. The spread bounces around in the 53-61 range while the mean rises, peaks, and starts to fall, and only drops noticeably (to 38) in the very last period. So a log transform doesn't have an obvious visual justification here, even though detrending clearly does.

References

- Frery, Alejandro C. 2026. *The Monte Carlo Method*. DATA301/DATA471 tutorial slides. Victoria University of Wellington.
- Hyndman, Rob J., and George Athanasopoulos. 2018. *Forecasting: Principles and Practice*. 2nd ed. OTexts. <https://OTexts.com/fpp2>.
- Hyndman, Rob J., and Yanan Fan. 1996. “Sample Quantiles in Statistical Packages.” *The American Statistician* 50 (4): 361–65. <https://doi.org/10.2307/2684934>.
- Wicklin, Rick. 2017. *Sample Quantiles: A Comparison of 9 Definitions*. SAS Blogs, The DO Loop. <https://blogs.sas.com/content/iml/2017/05/24/definitions-sample-quantiles.html>.

Appendix: Declaration of AI Use

Claude Code was used to help check and improve the R code for both exercises, to summarise and search reference material on Monte Carlo method applications, to act as a sounding board for discussing ideas, and to assist with Quarto/L^AT_EX typesetting and English language polishing. All written interpretations and analytical decisions are my own work.